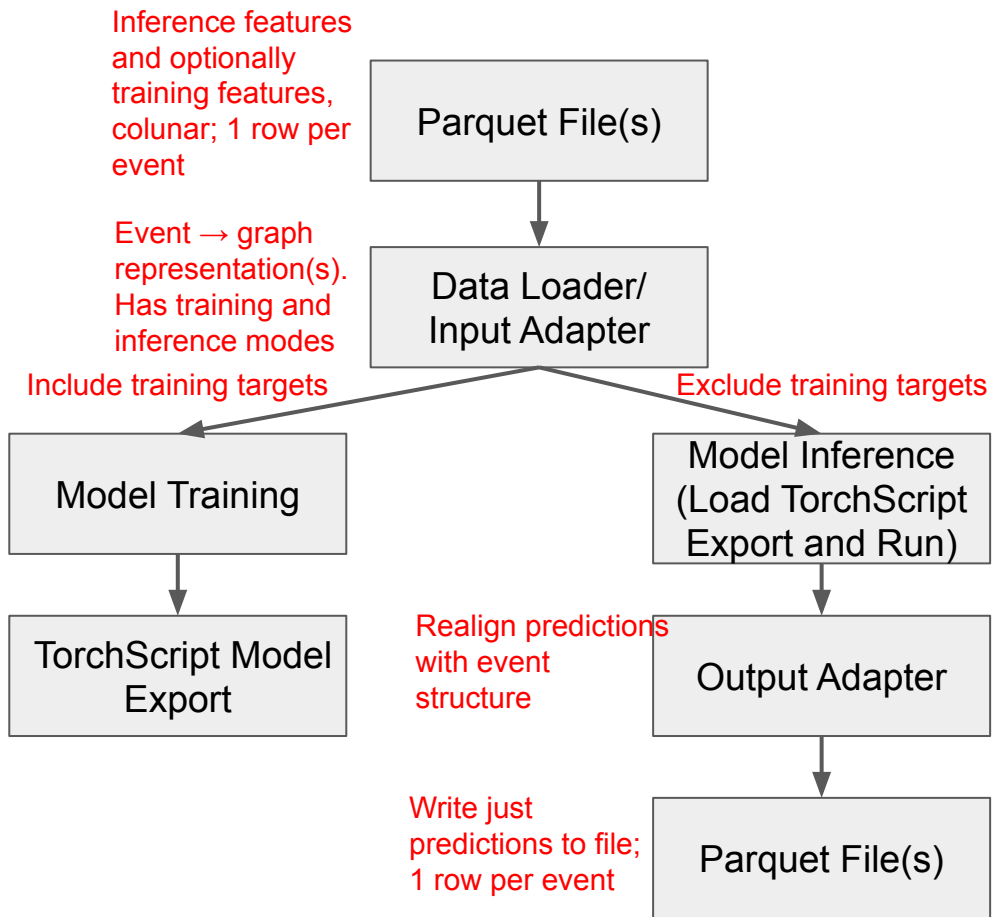


ML Scalable Pipeline Design

Jack Carlton
University of Kentucky

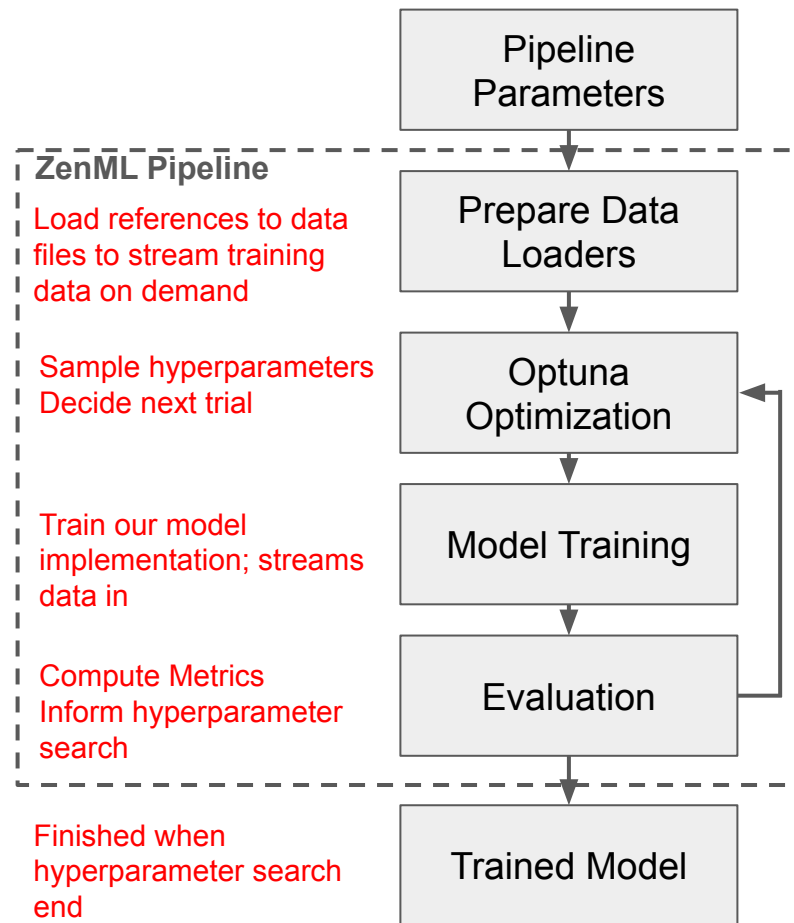
Data Loading

- **Problem:** We will eventually scale to large training data sets
 - It becomes impossible to store all data in RAM at once
 - Operating one event at a time is too slow
 - Doesn't take advantage of parallel data loading operations
- **Solution:** Stream data into and out of models
 - Load data in one chunk at a time
 - Happens parallel with model execution
 - Model requests data when done training on it's current micobatch
 - Write data predictions in chunks as well



Typical Training Pipeline

- [PyTorch](#)
 - Provides a standard base abstraction classes for many model types (ex. `nn.Module`)
 - Fast tensor operations for data loading
 - Avoids large python loops when making graphs from feature columns
- [ZenML](#)
 - Encodes pipelines as composable, declarative units and orchestrates execution
 - Allows pipelines to grow by adding or reordering steps; easy to add new pipelines
 - Manages pipeline state, artifacts, and execution metadata outside user code
- [Optuna](#)
 - Isolates hyperparameter search code
 - Enables experimentation without modifying core implementations



Simplified Example Pipeline for Training Models

Auxiliary Slides

= Not an ML stage

= An ML (trained) stage

Current AI/ML ATAR Reconstruction Pipeline

Simulation Pipeline: Create ML Ready Dataset

Monte Carlo +
Detector Response

Truth level descriptions
of entire events +
detector response;
optional event mixing

ML Ready Dataset
Gaudi Pipeline

Forms ML ready dataset
(parquet files, efficient
columnar storage)
Applies preprocessing
stages to derive additional
feature columns (ex. Time
group labels)

AI/ML Pipeline: Append predictions to events, each stage follows the following prescription: Event Data → Graphs → Model → Graph Level Predictions → Event Level Predictions

Group Classifier

Adds group level
particle classes

Group Splitter

Adds hit level
particle classes

Endpoint Regressor

Adds endpoint (x,y,z)
estimates

Pattern Finder

Adds affinities between
time group

Pion Stop
Regressor

Adds pion stops
(x,y,z) estimates

Positron Angle
Finder

Adds positron angles
($\sin\theta \cos\phi$, $\sin\theta \sin\phi$,
 $\cos\theta$) estimates

**Final Goal: Unify all stages into a single
end-to-end ML model**